

# Comparative Analysis of ML Models for Stroke Risk Prediction

Sharmeen Kapoorwala

Shrusti HC

March 15, 2026

## Step 1 - Determine Data and Variables

### Data Source

The dataset used for this project is **healthcare-dataset-stroke-data.csv** which is from kaggle, which contains patient health and demographic information used to predict whether a patient has experienced a stroke.

The dataset contains of 5110 observations (patients) and 12 variable (columns) describing patient characteristics.

### Target Variable

- Stroke - categorical (0 = No Stroke, 1 = Stroke) shows if the patient had experienced a stroke

### Input Variables

- gender
- age
- hypertension - if the patient has hypertension
- heart\_disease - if the patient has heart disease
- ever\_married - if the patient has ever been married
- work\_type - Type of employment
- Residence\_type - urban or rural
- avg\_glucose\_level - Average glucose level in blood
- bmi - Body Mass Index
- smoking\_status - if the patient smokes

### Remove

- id (identifier) - does not provide predictive behavior

## Step 2 -Load, Explore, and Clean the Data

In this step, the dataset is imported and examined for structure, missing values, duplicate records, and class distribution. The `id` variable is removed because it has no predictive meaning. The response variable and categorical predictors are converted to factor type so they can be used appropriately in classification models.

The dataset contains missing values in the `bmi` variable, so those missing values are replaced using the median BMI. This approach allows the full dataset to be retained while avoiding distortion from missing observations.

The distribution of the response variable is also examined. This is important because model performance should not be judged by accuracy alone in an imbalanced classification problem.

```
library(tidyverse)
library(tidymodels)
library(DataExplorer)
library(rsample)
library(nnet)
library(randomForest)
library(yardstick)
library(glmnet)
library(themis) #SMOTE (for imbalanced data)
library(discrim)
```

```
#load dataset
heart <- read.csv("healthcare-dataset-stroke-data.csv")

#Explore
glimpse(heart)
```

```
Rows: 5,110
Columns: 12
$ id          <int> 9046, 51676, 31112, 60182, 1665, 56669, 53882, 10434~
$ gender      <chr> "Male", "Female", "Male", "Female", "Female", "Male"~
$ age        <dbl> 67, 61, 80, 49, 79, 81, 74, 69, 59, 78, 81, 61, 54, ~
$ hypertension <int> 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1~
$ heart_disease <int> 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0~
$ ever_married <chr> "Yes", "Yes", "Yes", "Yes", "Yes", "Yes", "Yes", "No~
$ work_type   <chr> "Private", "Self-employed", "Private", "Private", "S~
$ Residence_type <chr> "Urban", "Rural", "Rural", "Urban", "Rural", "Urban"~
$ avg_glucose_level <dbl> 228.69, 202.21, 105.92, 171.23, 174.12, 186.21, 70.0~
```

```
$ bmi          <chr> "36.6", "N/A", "32.5", "34.4", "24", "29", "27.4", "~
$ smoking_status <chr> "formerly smoked", "never smoked", "never smoked", "~
$ stroke        <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
```

```
#remove ID column
heart$id <- NULL

#check duplicates
sum(duplicated(heart))
```

```
[1] 0
```

```
heart |> count(stroke) |> mutate(prop = n/sum(n)) #proportion for patients who had stroke
```

```
  stroke    n      prop
1      0 4861 0.95127202
2      1  249 0.04872798
```

```
heart$stroke <- factor(heart$stroke)
heart$bmi <- as.numeric(heart$bmi)
heart$gender <- as.factor(heart$gender)
heart$ever_married <- as.factor(heart$ever_married)
heart$work_type <- as.factor(heart$work_type)
heart$Residence_type <- as.factor(heart$Residence_type)
heart$smoking_status <- as.factor(heart$smoking_status)

#report
#create_report(heart)

#check missing values
colSums(is.na(heart)) #BMI has 201 missing values
```

```
      gender      age      hypertension      heart_disease
      0          0          0          0
ever_married work_type Residence_type avg_glucose_level
      0          0          0          0
      bmi smoking_status      stroke
      201          0          0
```

```
#Impute missing BMI with median  
heart$bmi[is.na(heart$bmi)] <- median(heart$bmi, na.rm = T)
```

## Step 3 - Develop Models

### Train/Test Split + Cross-Validation

Splitting 75% of the data for training and 25% for testing

```
set.seed(369)

data_split <- initial_split(heart, prop = 0.75, strata = stroke)
train_data <- training(data_split)
test_data <- testing(data_split)

cv_folds <- vfold_cv(train_data, v = 5, strata = stroke)
```

### Recipe

```
stroke_recipe <- recipe(stroke ~ ., data = train_data) |>
  step_impute_median(all_numeric_predictors()) |>
  step_impute_mode(all_nominal_predictors()) |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors()) |> #for KNN
  step_smote(stroke) |> #for imbalanced data
  step_zv(all_predictors())
```

### Null Model

```
null_mod <- glm(stroke ~ 1, data = train_data, family = "binomial")
summary(null_mod)
```

Call:

```
glm(formula = stroke ~ 1, family = "binomial", data = train_data)
```

Coefficients:

```
              Estimate Std. Error z value Pr(>|z|)
(Intercept) -2.97000    0.07497  -39.61   <2e-16 ***
---

```

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1494.2 on 3831 degrees of freedom  
Residual deviance: 1494.2 on 3831 degrees of freedom  
AIC: 1496.2

Number of Fisher Scoring iterations: 5

## Full Model

```
full_mod <- glm(stroke ~ ., data = train_data, family = "binomial")  
summary(full_mod)
```

Call:

```
glm(formula = stroke ~ ., family = "binomial", data = train_data)
```

Coefficients:

|                            | Estimate   | Std. Error | z value | Pr(> z )     |
|----------------------------|------------|------------|---------|--------------|
| (Intercept)                | -2.055e+01 | 4.683e+02  | -0.044  | 0.9650       |
| genderMale                 | -1.863e-02 | 1.647e-01  | -0.113  | 0.9099       |
| age                        | 7.717e-02  | 6.826e-03  | 11.305  | < 2e-16 ***  |
| hypertension               | 4.059e-01  | 1.907e-01  | 2.128   | 0.0333 *     |
| heart_disease              | 1.062e-01  | 2.239e-01  | 0.475   | 0.6351       |
| ever_marriedYes            | -2.177e-01 | 2.627e-01  | -0.829  | 0.4073       |
| work_typeGovt_job          | 1.259e+01  | 4.683e+02  | 0.027   | 0.9785       |
| work_typeNever_worked      | -6.344e-01 | 3.415e+03  | 0.000   | 0.9999       |
| work_typePrivate           | 1.284e+01  | 4.683e+02  | 0.027   | 0.9781       |
| work_typeSelf-employed     | 1.250e+01  | 4.683e+02  | 0.027   | 0.9787       |
| Residence_typeUrban        | -2.492e-02 | 1.604e-01  | -0.155  | 0.8765       |
| avg_glucose_level          | 5.619e-03  | 1.390e-03  | 4.041   | 5.31e-05 *** |
| bmi                        | -3.181e-03 | 1.331e-02  | -0.239  | 0.8112       |
| smoking_statusnever smoked | -1.515e-01 | 2.050e-01  | -0.739  | 0.4597       |
| smoking_statussmokes       | 8.384e-02  | 2.574e-01  | 0.326   | 0.7446       |
| smoking_statusUnknown      | -2.487e-02 | 2.393e-01  | -0.104  | 0.9172       |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1494.2 on 3831 degrees of freedom  
Residual deviance: 1159.5 on 3816 degrees of freedom  
AIC: 1191.5

Number of Fisher Scoring iterations: 18



## Step 4 - Model classifications

### Models used for fitting:

- Logistic Regression
- kNN
- Naive Bayes
- Decision Tree
- Random Forest
- Boost C5.0

```
lr_mod <- logistic_reg() |> set_engine("glm")

knn_mod <- nearest_neighbor(neighbors = 5) |>
  set_engine("kknn") |>
  set_mode("classification")

dt_mod <- decision_tree() |>
  set_engine("rpart") |>
  set_mode("classification")

rf_mod <- rand_forest( trees = 500) |>
  set_engine("ranger") |>
  set_mode("classification")

nb_mod <- naive_Bayes() |>
  set_engine("naivebayes") |>
  set_mode("classification")

boost_mod <- boost_tree() |>
  set_engine("C5.0") |>
  set_mode("classification")
```

### Workflows

```
lr_wf <- workflow() |> add_model(lr_mod) |> add_recipe(stroke_recipe)

knn_wf <- workflow() |> add_model(knn_mod) |> add_recipe(stroke_recipe)

dt_wf <- workflow() |> add_model(dt_mod) |> add_recipe(stroke_recipe)
```

```
rf_wf <- workflow() |> add_model(rf_mod) |> add_recipe(stroke_recipe)

nb_wf <- workflow() |> add_model(nb_mod) |> add_recipe(stroke_recipe)

boost_wf <- workflow() |> add_model(boost_mod) |> add_recipe(stroke_recipe)
```

## Fit Models

### Logistic Regression Model

- The Logistic Regression Model achieved **76% accuracy**, showing moderate overall prediction performance. However, the **low kappa value (0.154)** indicates weak agreement beyond chance, suggesting limited reliability in predicting stroke cases.

```
lr_fit <- fit(lr_wf, data = train_data)

lr_pred <- predict(lr_fit, test_data) |>
  bind_cols(test_data)

metrics <- metric_set(accuracy, kap)

lr_met <- metrics(lr_pred, truth = stroke, estimate = .pred_class) |>
  mutate(model = "Logistic Regression Model")

lr_met
```

```
# A tibble: 2 x 4
  .metric .estimator .estimate model
  <chr>    <chr>      <dbl> <chr>
1 accuracy binary      0.760 Logistic Regression Model
2 kap      binary      0.154 Logistic Regression Model
```

```
conf_mat(lr_pred, truth = stroke, estimate = .pred_class)
```

|            | Truth |    |
|------------|-------|----|
| Prediction | 0     | 1  |
| 0          | 927   | 18 |
| 1          | 289   | 44 |

## kNN Model

- The kNN Model achieved **85.8% accuracy**, indicating good overall classification performance. However, the **very low kappa value (0.0247)** suggests the model has almost no agreement beyond chance, meaning it poorly identifies stroke cases despite high accuracy.

```
knn_fit <- fit(knn_wf, data = train_data)

knn_pred <- predict(knn_fit, test_data) |>
  bind_cols(test_data)

metrics <- metric_set(accuracy, kap)

knn_met <- metrics(knn_pred, truth = stroke, estimate = .pred_class) |>
  mutate(model = "kNN Model")

knn_met
```

```
# A tibble: 2 x 4
  .metric .estimator .estimate model
  <chr>   <chr>      <dbl> <chr>
1 accuracy binary      0.858 kNN Model
2 kap     binary      0.0247 kNN Model
```

```
conf_mat(knn_pred, truth = stroke, estimate = .pred_class)
```

|            | Truth |    |
|------------|-------|----|
| Prediction | 0     | 1  |
| 0          | 1087  | 53 |
| 1          | 129   | 9  |

## Decision Tree Model

- A Decision Tree model achieved **69.6% accuracy**, indicating moderate prediction performance. The **Kappa value of 0.120** suggests weak agreement beyond chance, showing the model has limited reliability in predicting stroke cases.

```
dt_fit <- fit(dt_wf, data = train_data)

dt_pred <- predict(dt_fit, test_data) |>
  bind_cols(test_data)

metrics <- metric_set(accuracy, kap)

dt_met <- metrics(dt_pred, truth = stroke, estimate = .pred_class) |>
  mutate(model = "Decision Tree Model")

dt_met
```

```
# A tibble: 2 x 4
  .metric .estimator .estimate model
  <chr>    <chr>      <dbl> <chr>
1 accuracy binary      0.696 Decision Tree Model
2 kap      binary      0.120 Decision Tree Model
```

```
conf_mat(dt_pred, truth = stroke, estimate = .pred_class)
```

|            | Truth |    |
|------------|-------|----|
| Prediction | 0     | 1  |
| 0          | 842   | 15 |
| 1          | 374   | 47 |

## Random Forest Model

- Random Forest Model achieved **88.2% accuracy**, showing strong overall classification performance. However, the **low Kappa value (0.069)** indicates weak agreement beyond chance, suggesting poor detection of stroke cases.

```
rf_fit <- fit(rf_wf, data = train_data)

rf_pred <- predict(rf_fit, test_data) |>
  bind_cols(test_data)

metrics <- metric_set(accuracy, kap)

rf_met <- metrics(rf_pred, truth = stroke, estimate = .pred_class) |>
  mutate(model = "Random Forest Model")
```

```
rf_met
```

```
# A tibble: 2 x 4
  .metric .estimator .estimate model
  <chr>    <chr>         <dbl> <chr>
1 accuracy binary         0.882 Random Forest Model
2 kap      binary         0.0692 Random Forest Model
```

```
conf_mat(rf_pred, truth = stroke, estimate = .pred_class)
```

|            | Truth |    |
|------------|-------|----|
| Prediction | 0     | 1  |
| 0          | 1116  | 51 |
| 1          | 100   | 11 |

## Naive Bayes Model

- Naive Bayes Model achieved **53.8% accuracy**, indicating relatively low overall prediction performance. The **Kappa value (0.073)** shows weak agreement beyond chance, suggesting the model is not reliable for predicting stroke cases.

```
nb_fit <- fit(nb_wf, data = train_data)

nb_pred <- predict(nb_fit, test_data) |>
  bind_cols(test_data)

metrics <- metric_set(accuracy, kap)

nb_met <- metrics(nb_pred, truth = stroke, estimate = .pred_class) |>
  mutate(model = "Naive Bayes Model")

nb_met
```

```
# A tibble: 2 x 4
  .metric .estimator .estimate model
  <chr>    <chr>         <dbl> <chr>
1 accuracy binary         0.538 Naive Bayes Model
2 kap      binary         0.0728 Naive Bayes Model
```

```
conf_mat(nb_pred, truth = stroke, estimate = .pred_class)
```

|            | Truth |    |
|------------|-------|----|
| Prediction | 0     | 1  |
| 0          | 634   | 8  |
| 1          | 582   | 54 |

## Boost C5.0 Model

- The Boosted C5.0 model achieved **92.4% accuracy**, indicating strong overall prediction performance. However, the **Kappa value (0.131)** shows weak agreement beyond chance, suggesting limited reliability in detecting stroke cases.

```
boost_fit <- fit(boost_wf, data = train_data)

boost_pred <- predict(boost_fit, test_data) |>
  bind_cols(test_data)

metrics <- metric_set(accuracy, kap)

boost_met <- metrics(boost_pred, truth = stroke, estimate = .pred_class) |>
  mutate(model = "Boost C5.0 Model")

boost_met
```

```
# A tibble: 2 x 4
  .metric .estimator .estimate model
  <chr>    <chr>         <dbl> <chr>
1 accuracy binary         0.924 Boost C5.0 Model
2 kap      binary         0.131 Boost C5.0 Model
```

```
conf_mat(boost_pred, truth = stroke, estimate = .pred_class)
```

|            | Truth |    |
|------------|-------|----|
| Prediction | 0     | 1  |
| 0          | 1171  | 52 |
| 1          | 45    | 10 |

## Best Model without Tuning

After comparing the six classification models, the models show relatively high accuracy but low kappa values, indicating that while many predictions are correct, the models struggle to reliably identify stroke cases due to class imbalance.

Among the models, **Boost C5.0 model** achieved the highest accuracy, but none of the models demonstrated strong agreement beyond chance, suggesting that further tuning or balancing of the dataset may be needed to improve stroke prediction.

```
all_metrics <- bind_rows(lr_met,knn_met,rf_met,dt_met, boost_met,nb_met) |>
  select(model,.metric, .estimate)

met_table <- all_metrics |>
  pivot_wider(names_from = .metric, values_from = .estimate) |>
  arrange(desc(accuracy))

met_table
```

```
# A tibble: 6 x 3
  model                accuracy    kap
  <chr>                <dbl>  <dbl>
1 Boost C5.0 Model    0.924 0.131
2 Random Forest Model 0.882 0.0692
3 kNN Model           0.858 0.0247
4 Logistic Regression Model 0.760 0.154
5 Decision Tree Model 0.696 0.120
6 Naive Bayes Model   0.538 0.0728
```

```
best_wf <- workflow() |>
  add_model(boost_mod) |>
  add_recipe(stroke_recipe)

best_fit <- fit(best_wf, data = train_data)
```

## Step 5 - Tuning with Cross Validation

### Fit Models with Cross-Validation

Since the response variable is imbalanced, using multiple metrics gives a better measure of performance than accuracy alone. The cross-validation results show that **C5.0 (Boosted)** model performed the best overall with the **highest accuracy (0.92), F-measure (0.96), and recall (0.96)**. The **Random Forest** model also performed well with **high accuracy (0.90) and precision (0.96)**.

Overall, tree-based ensemble models (C5.0 and Random Forest) outperformed the other models, while Naive Bayes showed the weakest performance, indicating it is less suitable for this dataset.

```
lr_res <- fit_resamples(lr_wf, cv_folds,
                        metrics = metric_set(accuracy, precision, recall, f_meas, roc_auc))

knn_res <- fit_resamples(knn_wf, cv_folds,
                        metrics = metric_set(accuracy, precision, recall, f_meas, roc_auc))

dt_res <- fit_resamples(dt_wf, cv_folds,
                        metrics = metric_set(accuracy, precision, recall, f_meas, roc_auc))

rf_res <- fit_resamples(rf_wf, cv_folds,
                        metrics = metric_set(accuracy, precision, recall, f_meas, roc_auc))

nb_res <- fit_resamples(nb_wf, cv_folds,
                        metrics = metric_set(accuracy, precision, recall, f_meas, roc_auc))

boost_res <- fit_resamples(boost_wf, cv_folds,
                           metrics = metric_set(accuracy, precision, recall, f_meas, roc_auc))
```

### Compare Models

```
results <- bind_rows(
  lr_res |> collect_metrics() |> mutate(model = "Logistic"),
  knn_res |> collect_metrics() |> mutate(model = "kNN"),
  dt_res |> collect_metrics() |> mutate(model = "Decision Tree"),
  rf_res |> collect_metrics() |> mutate(model = "Random Forest"),
  nb_res |> collect_metrics() |> mutate(model = "Naive Bayes"),
```



```
boost_res |> collect_metrics() |> mutate(model = "C5.0")
)
```

```
#results
```

```
res_clean <- results |>
  select(model, .metric, mean) |>
  mutate(mean = round(mean, 2)) |>
  pivot_wider(names_from = .metric, values_from = mean) |>
  arrange(desc(accuracy))
```

```
res_clean
```

```
# A tibble: 6 x 6
```

|   | model         | accuracy | f_meas | precision | recall | roc_auc |
|---|---------------|----------|--------|-----------|--------|---------|
|   | <chr>         | <dbl>    | <dbl>  | <dbl>     | <dbl>  | <dbl>   |
| 1 | C5.0          | 0.92     | 0.96   | 0.95      | 0.96   | 0.78    |
| 2 | Random Forest | 0.9      | 0.95   | 0.96      | 0.94   | 0.81    |
| 3 | kNN           | 0.85     | 0.92   | 0.96      | 0.89   | 0.64    |
| 4 | Logistic      | 0.75     | 0.85   | 0.98      | 0.75   | 0.85    |
| 5 | Decision Tree | 0.71     | 0.82   | 0.99      | 0.71   | 0.79    |
| 6 | Naive Bayes   | 0.51     | 0.65   | 1         | 0.49   | 0.83    |

## Predict on Test Dataset

Using the best model, which is **C5.0**, on the test dataset, produces 1223 predictions for non-stroke and 55 predictions for stroke, and the results were saved to a **submission.csv** file.

```
test <- test_data |>
  mutate(
    bmi = as.numeric(bmi),
    gender = factor(gender),
    ever_married = factor(ever_married),
    work_type = factor(work_type),
    Residence_type = factor(Residence_type),
    smoking_status = factor(smoking_status)
  )

test_pred <- predict(best_fit, test, type = "class") #best fit is C5.0

#creating submission
```

```
submission <- tibble(
  Stroke = test_pred$.pred_class
)

table(test_pred$.pred_class)
```

```

  0    1
1223  55
```

```
#save into .csv file
write.csv(submission, "submission.csv")
```

## ROC Curve

- The ROC plot shows how well each model separates stroke and non-stroke cases at different cutoff values. In general, the curves for Logistic Regression and Random Forest stay higher than most of the others, which suggests they do a better job distinguishing between the two classes. kNN appears to perform the weakest because its curve is closer to the diagonal line, meaning its predictions are less effective at separating the classes.

The C5.0 model still performs fairly well, even though its ROC curve is not the strongest overall. This means that while C5.0 may not be the best model based only on ROC performance, it can still be considered the best overall model if we focus more on accuracy, recall, and F1 score, where it performed very strongly.

```
knn_prob <- predict(knn_fit, test_data, type = "prob") |>
  bind_cols(test_data)

boost_prob <- predict(boost_fit, test_data, type = "prob") |>
  bind_cols(test_data)

lr_prob <- predict(lr_fit, test_data, type = "prob") |>
  bind_cols(test_data)

rf_prob <- predict(rf_fit, test_data, type = "prob") |>
  bind_cols(test_data)

nb_prob <- predict(nb_fit, test_data, type = "prob") |>
  bind_cols(test_data)
```

```

dt_prob <- predict(dt_fit, test_data, type = "prob") |>
  bind_cols(test_data)

knn_roc <- roc_curve(knn_prob, truth = stroke, .pred_1) |>
  mutate(model = "kNN")

boost_roc <- roc_curve(boost_prob, truth = stroke, .pred_1) |>
  mutate(model = "C5.0")

rf_roc <- roc_curve(rf_prob, truth = stroke, .pred_1) |>
  mutate(model = "Random Forest")

lr_roc <- roc_curve(lr_prob, truth = stroke, .pred_1) |>
  mutate(model = "Logistic Regression")

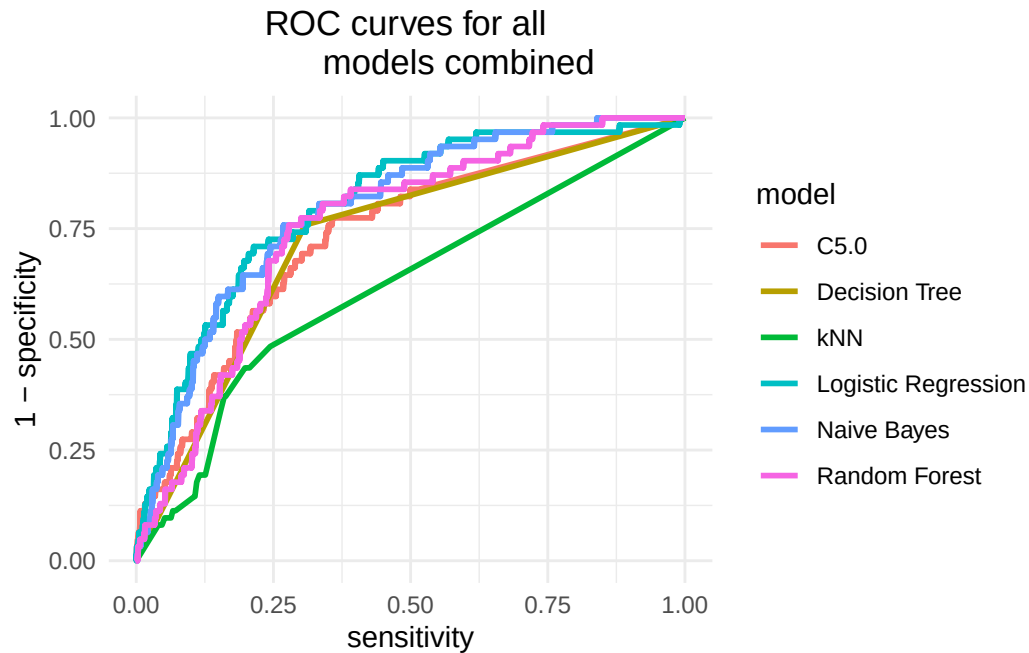
nb_roc <- roc_curve(nb_prob, truth = stroke, .pred_1) |>
  mutate(model = "Naive Bayes")

dt_roc <- roc_curve(dt_prob, truth = stroke, .pred_1) |>
  mutate(model = "Decision Tree")

all_roc <- bind_rows(knn_roc, dt_roc, boost_roc, rf_roc, lr_roc, nb_roc)

ggplot(all_roc, aes(y = 1 - specificity, x = sensitivity, color = model)) +
  geom_line(size = 1) +
  ggtitle("ROC curves for all
          models combined") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))

```



## Conclusion

In conclusion, the Boost C5.0 model was selected as the best model for predicting stroke. It achieved the highest test accuracy (0.924) and also had the strongest cross-validated performance, with the highest accuracy (0.92), F1 score (0.96), and recall (0.96) among all models considered. These results indicate that C5.0 provided the best overall classification performance for this dataset and was therefore chosen as the final model.